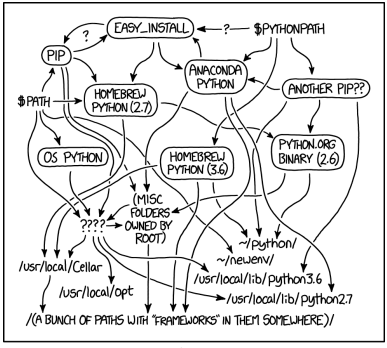
Docker: основные команды

Селиверстов А., Слесарева Е.

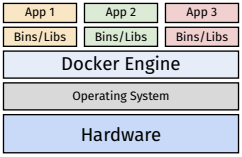
Санкт-Петербургский политехнический университет Петра Великого
Группа 5040102/60101

31 октября 2026 г.

Введение в Docker: works well for me!



MY PYTHON ENVIRONMENT HAS BECOME SO DEGRADED
THAT MY LAPTOP HAS BEEN DECLARED A SUPERFUND SITE.



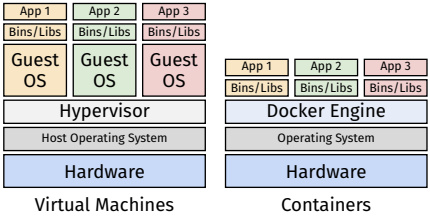
Containers

Docker упаковывает программу вместе с окружением в образ и запускает её в контейнере: изолированном процессе на общем ядре ОС.

ОС	Где работает контейнер
Linux	прямо на ядре хоста как обычный процесс. Docker Engine изолирует его (namespaces) и ограничивает ресурсы (cgroups)
macOS	в Linux-ВМ, которую поднимает Docker Desktop
Windows	в Linux-ВМ, обычно через WSL 2
Другой процессор	образ amd64 на arm64 (Mac с Apple): команды эмулируются, это медленнее

Рисунки: xkcd 1987, Randall Munroe, CC BY-NC 2.5; схема по мотивам Docker vs VM, Drinkler, CC BY-SA 4.0.
Документация: Docker overview, Engine security, Docker Desktop VMM

Контейнер vs VM



	Виртуальная машина	**Контейнер**
ОС	своя гостевая ОС со своим ядром	ядро хоста
Размер образа	десятки ГБ	десятки МБ
Запуск	медленно: загружается ОС	быстро: один процесс
Память	гостевая ОС занимает память всегда	только программа
Процессор	расходы на гипервизор и гостевую ОС	как у обычного процесса
Изоляция	сильнее: отдельное ядро	слабее: ядро общее

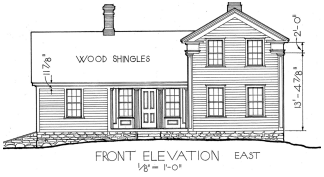
Схема по мотивам Docker vs VM, Drinkler, CC BY-SA 4.0, Wikimedia Commons.
Документация: docker.com: What is a container?

Введение

Реестр (registry)

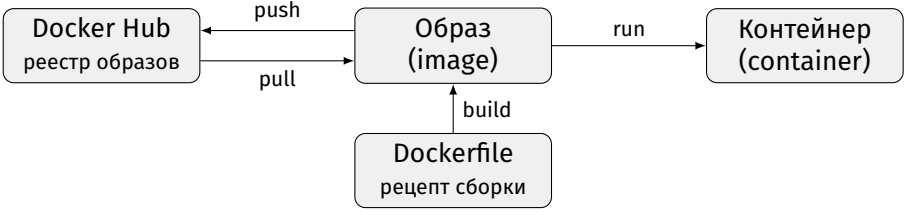


Образ (image)



Контейнер (container)





Рисунки: чертёж, фото дома (Historic American Buildings Survey), архив чертежей (Architect of the Capitol); Wikimedia Commons.
Документация: Docker overview, раздел Docker objects

Анатомия команды

```
docker <объект> <действие> [флаги] [аргументы]
```

тип объекта:
container, image,
volume, network

что сделать:
run, ls, rm, logs

настройки:
`-d`, `-p 8080:80`

иногда опускается:
`docker run` =
`docker container run`

help!!

zsh

```
$ docker --help

Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers

Common Commands:
  run         Create and run a new container from an image
  exec        Execute a command in a running container
  ps          List containers
  build       Build an image from a Dockerfile
  pull        Download an image from a registry
  push        Upload an image to a registry
  images      List images
 ...
```

zsh

```
$ docker container run -d --name web -p 8080:80 \
    -e TZ=Europe/Moscow -v web_data:/data \
    --restart unless-stopped --help nginx:1.27

Usage:  docker container run [OPTIONS] IMAGE [COMMAND] [ARG...]

Create and run a new container from an image

Aliases:
  docker container run, docker run

Options:
 ...
```

zsh

```
$ docker info

Client: Docker Engine - Community
 Version:    27.5.1
 Context:    default
 Debug Mode: false
 Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.20.1
    Path:     ~/.docker/cli-plugins/docker-buildx
  compose: Docker Compose (Docker Inc.)
    Version:  2.33.0
    Path:     ~/.docker/cli-plugins/docker-compose

 ...
```

zsh

```
$ docker buildx imagetools create --help

Create a new image based on source images

Usage:  docker buildx imagetools create [OPTIONS] [SOURCE] [SOURCE...]

Create a new image based on source images

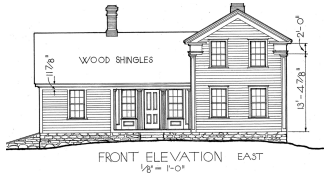
Options:
      --annotation stringArray   Add annotation to the image
      --append                   Append to existing manifest
      --builder string           Override the configured builder instance
  -D, --debug                    Enable debug logging
      --dry-run                  Show final image instead of pushing
 ...
```

Документация: справочник Docker CLI, `docker system info`

Реестр (registry)

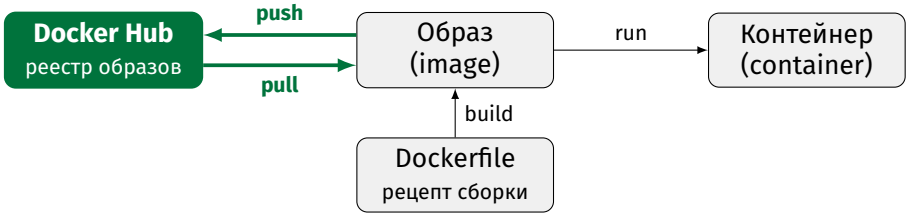


Образ (image)



Контейнер (container)





Рисунки: чертёж, фото дома (Historic American Buildings Survey), архив чертежей (Architect of the Capitol); Wikimedia Commons.
Документация: Docker overview, раздел Docker objects

Реестр образов

zsh

```
$ docker login -u username
# сторонний реестр, например GitHub:
$ docker login ghcr.io -u username
```

```
Password:
Login Succeeded
```

zsh

```
$ docker search --limit 5 nginx
```

```
NAME                              DESCRIPTION                                     STARS   OFFICIAL
nginx                             Official build of Nginx.                        21414   [OK]
nginx/nginx-ingress               NGINX and  NGINX Plus Ingress Controllers fo…  124
nginx/nginx-prometheus-exporter   NGINX Prometheus Exporter for NGINX and NGIN…  53
nginx/nginx-ingress-operator      NGINX Ingress Operator for NGINX and NGINX P…  5
nginx/nginx-unprivileged          Unprivileged NGINX images                       2
```

zsh

```
$ docker pull hello-world
```

```
Using default tag: latest
latest: Pulling from library/hello-world
58dee6a49ef1: Pull complete
Digest: sha256:c567b231f65614688711a75f31518a064258a51d442117a3e771a3a8fdba78ef
Status: Downloaded newer image for hello-world:latest
docker.io/library/hello-world:latest
```

zsh

```
$ docker tag hello-cpp ghcr.io/username/hello-cpp:1.0
$ docker push ghcr.io/username/hello-cpp:1.0
```

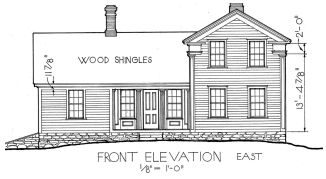
```
The push refers to repository [ghcr.io/username/hello-cpp]
52889952fb5c: Pushed
44096d18eb80: Pushed
f8a9e9ebe1c3: Pushed
107ffca6dcd3: Pushed
1.0: digest: sha256:226045f89c89a151812927b834a69496df2b90d525b451e332ad2fd5c7659693
```

Документация: What is a registry?, GitHub Container Registry

Реестр (registry)

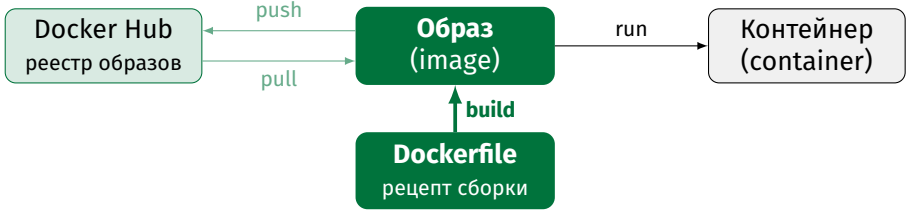


Образ (image)



Контейнер (container)





Рисунки: чертёж, фото дома (Historic American Buildings Survey), архив чертежей (Architect of the Capitol); Wikimedia Commons.
Документация: Docker overview, раздел Docker objects

Dockerfile

Dockerfile 1

```
FROM alpine:3.20
RUN apk add --no-cache curl
CMD ["curl", "-s", "wttr.in/Saint-Petersburg?format=3"]
```

Dockerfile 2

```
FROM python:3.15-slim
RUN pip install --no-cache-dir jupyterlab pandas numpy matplotlib
EXPOSE 8888
CMD ["jupyter", "lab", "--ip=0.0.0.0", "--allow-root", "--no-browser"]
```

Dockerfile 3

```
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y g++
COPY main.cpp .
RUN g++ -O2 -o /usr/local/bin/app main.cpp
CMD ["app"]
```

Документация: Dockerfile reference

Сборка образа

zsh

```
$ docker image build -f Dockerfile1 -t weather .

[+] Building 1.7s (7/7) FINISHED
 ⇒ [internal] load build definition from Dockerfile1                    0.0s
 ⇒ [internal] load metadata for docker.io/library/alpine:3.20           1.5s
 ⇒ [internal] load .dockerignore                                        0.0s
 ⇒ [1/2] FROM docker.io/library/alpine:3.20@sha256:d9e853e87e55526f6b291  0.0s
 ⇒ CACHED [2/2] RUN apk add --no-cache curl                             0.0s
 ⇒ exporting to image                                                   0.2s
 ⇒ ⇒ naming to docker.io/library/weather:latest                         0.0s
```

zsh

```
$ docker image build -f Dockerfile2 -t jupyter-lab:1.0 .

[+] Building 83.9s (7/7) FINISHED
 ⇒ [internal] load build definition from Dockerfile2                    0.0s
 ⇒ [internal] load metadata for docker.io/library/python:3.15-slim      0.5s
 ⇒ [internal] load .dockerignore                                        0.0s
 ⇒ CACHED [1/2] FROM docker.io/library/python:3.15-slim@sha256:94d3426f1  0.0s
 ⇒ [2/2] RUN pip install --no-cache-dir jupyterlab pandas numpy matplot  63.3s
 ⇒ exporting to image                                                  20.0s
 ⇒ ⇒ naming to docker.io/library/jupyter-lab:1.0                        0.0s
```

zsh

```
$ docker image build -f Dockerfile3 -t hello-cpp .

[+] Building 32.5s (10/10) FINISHED
 ⇒ [internal] load build definition from Dockerfile3                    0.0s
 ⇒ [internal] load metadata for docker.io/library/ubuntu:24.04          0.8s
 ⇒ [internal] load .dockerignore                                        0.0s
 ⇒ CACHED [1/4] FROM docker.io/library/ubuntu:24.04@sha256:fe8156090fae  0.0s
 ⇒ [internal] load build context                                        0.0s
 ⇒ [2/4] RUN apt-get update && apt-get install -y g++                  18.1s
 ⇒ [3/4] COPY main.cpp .                                                0.0s
 ⇒ [4/4] RUN g++ -O2 -o /usr/local/bin/app main.cpp                     0.2s
 ⇒ exporting to image                                                  13.2s
 ⇒ ⇒ naming to docker.io/library/hello-cpp:latest                       0.0s
```

zsh

```
$ docker image ls

REPOSITORY    TAG       IMAGE ID       CREATED          SIZE
weather       latest    f297c446beca   17 minutes ago   15.1MB
hello-cpp     latest    0b72ca626817   16 minutes ago   411MB
jupyter-lab   1.0       c6c8abc3f876   15 minutes ago   626MB
```

Документация: `docker image build`, `docker image ls`

Удаление образов

zsh

```
$ docker image rm weather
```

```
Untagged: weather:latest
Deleted: sha256:f297c446beca2c04c9f48d9f5c6323fd83d93a757a0a39b2e72a198bc5fb5e60
```

zsh

```
$ docker image prune
```

```
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Deleted Images:
deleted: sha256:f1abbf53fcfcbe1a4f897b7b14fca3a12821b214d31cfa0461b52891585c14fa
deleted: sha256:0dfc47446c049b866fa3983f20fce6e85700e851951105c00c2f7cc24715dd2e

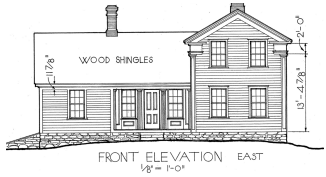
Total reclaimed space: 1.036GB
```

Документация: `docker image rm`, `docker image prune`

Реестр (registry)

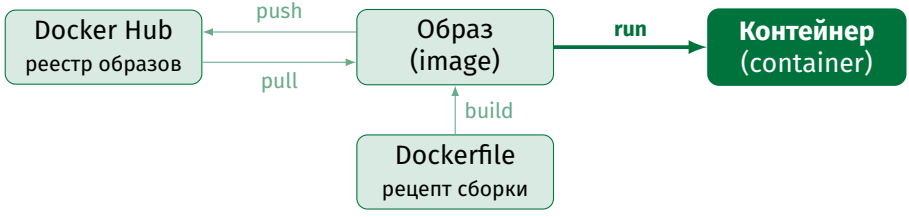


Образ (image)



Контейнер (container)





Рисунки: чертёж, фото дома (Historic American Buildings Survey), архив чертежей (Architect of the Capitol); Wikimedia Commons.
Документация: Docker overview, раздел Docker objects

Запуск контейнера

zsh

```
$ docker container run hello-cpp
```

```
Hello from C++ in Docker!
```

zsh

```
$ docker container run -it --rm hello-cpp bash
```

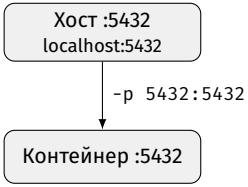
```
root@5ef02ef8bb0f:/# app
Hello from C++ in Docker!
root@5ef02ef8bb0f:/# exit
exit
```

Документация: `docker container run`

Запуск контейнера 2

Пример из руководства по PostgreSQL:

```
docker container run --rm --name postgres-dev \
  -e POSTGRES_PASSWORD=mysecretpassword \
  -p 5432:5432 \
  -v postgres_data:/var/lib/postgresql \
  -d postgres:18
```



Флаг	Что значит
`--rm`	удалить контейнер, когда он остановится
`--name`	задать имя контейнера
`-e KEY=value`	задать переменную окружения
`-p хост:контейнер`	опубликовать порт контейнера на хосте
`-v том:/путь`	подключить том, чтобы данные пережили контейнер
`-d`	запустить в фоне
`-it`	интерактивная сессия: `-i` держит STDIN открытым, `-t` даёт терминал

Документация: `--publish`, `--env`, руководство по PostgreSQL

Порядок флагов важен

Всё, что стоит после имени образа, Docker не разбирает, а отдаёт внутрь контейнера как команду, которую надо выполнить.



Документация: Running containers

Список контейнеров

zsh

```
$ docker ps
```

```
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS                    NAMES
4c13cc32f9e2   nginx:1.27     "/docker-entrypoint.…"   2 seconds ago   Up 2 seconds   0.0.0.0:8080→80/tcp      web
851ad429b798   postgres:17    "docker-entrypoint.s…"   2 seconds ago   Up 2 seconds   0.0.0.0:5432→5432/tcp    postgres-dev
```

zsh

```
$ docker ps --all
```

```
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS                     PORTS                    NAMES
4c13cc32f9e2   nginx:1.27     "/docker-entrypoint.…"   2 seconds ago   Up 2 seconds               0.0.0.0:8080→80/tcp      web
851ad429b798   postgres:17    "docker-entrypoint.s…"   2 seconds ago   Up 2 seconds               0.0.0.0:5432→5432/tcp    postgres-dev
e608607758a2   hello-cpp      "app"                    3 seconds ago   Exited (0) 2 seconds ago                            hello
```

Документация: `docker container ls`

Жизненный цикл контейнера

```
zsh
$ docker stop web
web
$ docker start web
web
$ docker stop web
web
$ docker rm web
web
$ docker rm --force postgres-dev
postgres-dev
```

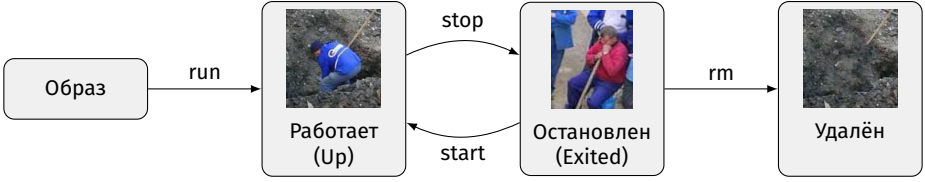
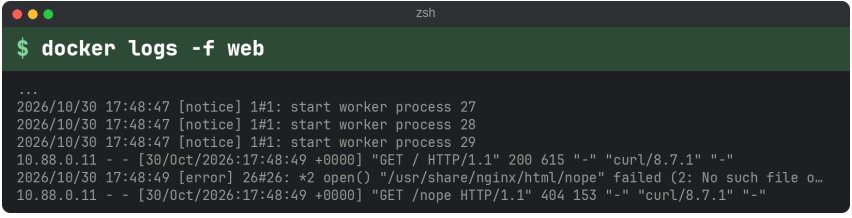


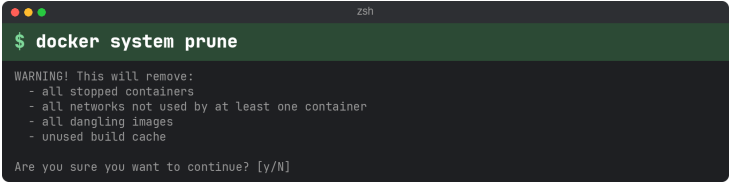
Фото: фрагменты снимка с evolo.eu (мем «single worker digging hole»).
Документация: `docker container`

Посмотреть логи



Документация: `docker container logs`

Уборка



Документация: `docker system prune`

Всё вместе

Сценарий из официального учебника Get started: Run an app. Посередине добавлены команды проверки.

zsh

```
# 1. скачать образ
$ docker pull docker/welcome-to-docker

Using default tag: latest
latest: Pulling from docker/welcome-to-docker
...
Digest: sha256:c4d56c24da4f009ecf8352146b43497fe78953edb4c679b841732beb97e588b0
Status: Downloaded newer image for docker/welcome-to-docker:latest

# 2. запустить в фоне, затем открыть localhost:8080
$ docker container run --detach --name welcome --publish 8080:80 docker/welcome-to-docker

cfc81e91ebf3c7d0f55da2732ea14b4ab5b853cc5427db7b706385050b1b9fd6a6

# 3. проверить: контейнер в списке, логи пишутся
$ docker ps

CONTAINER ID   IMAGE                     COMMAND                  CREATED         STATUS         PORTS                  NAMES
cfc81e91ebf3   docker/welcome-to-docker  "/docker-entrypoint.…"   2 seconds ago   Up 2 seconds   0.0.0.0:8080→80/tcp    welcome

$ docker logs welcome

...
10.88.0.15 - - [30/Oct/2026:17:49:01 +0000] "GET / HTTP/1.1" 200 651 "-" "curl/8.7.1" "-"

# 4. остановить и удалить
$ docker rm --force welcome

welcome
```

Полезные ссылки

1. Docker CLI Cheat Sheet (основа презентации)
2. Справочник Docker CLI
3. Get started: Run an app
4. Руководство по PostgreSQL
5. Docker overview
6. dockerd
7. docker image prune
8. docker container run
9. docker system prune